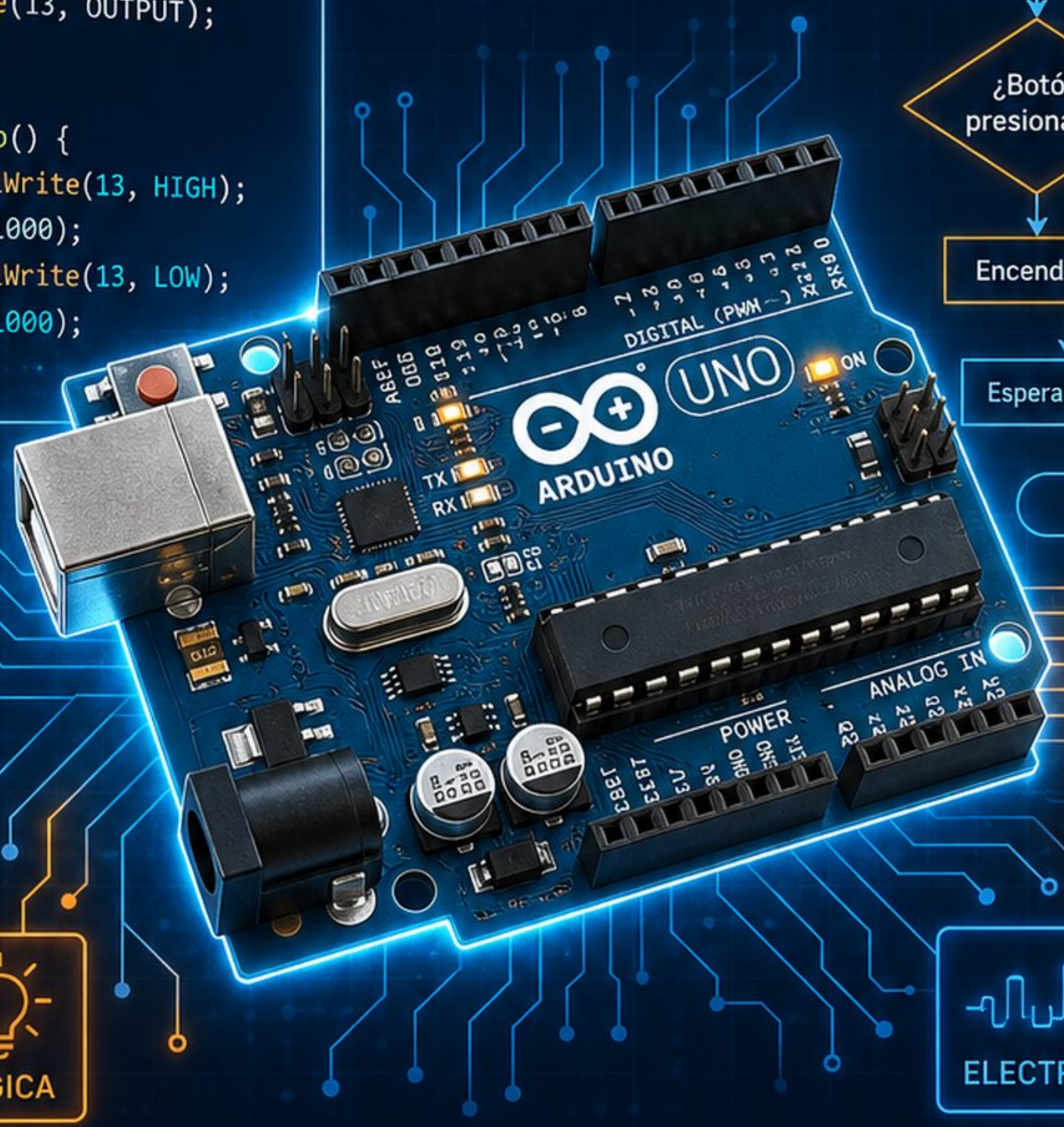
Programación Arduino:

Guía Visual Paso a Paso

Lógica y código desde cero

```
1  void setup() {
2    pinMode(13, OUTPUT);
3  }
4
5  void loop() {
6    digitalWrite(13, HIGH);
7    delay(1000);
8    digitalWrite(13, LOW);
9    delay(1000);
10 }
```





CÓDIGO

LÓGICA

ELECTRÓNICA

EXPLICACIONES CLARAS

IMÁGENES PASO A PASO

EJEMPLOS PRÁCTICOS

IDEAL PARA PRINCIPIANTES

APRENDE • COMPRENDE • CREA

Bienvenido

Aprende a programar Arduino paso a paso

Tabla de contenido



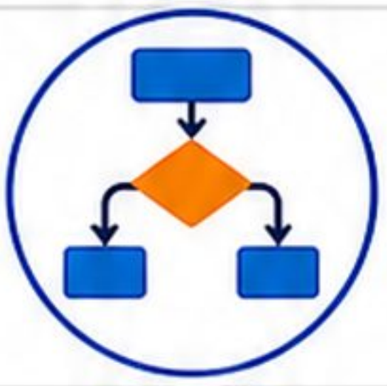
1. Estructura de un programa



2. Variables y tipos de datos



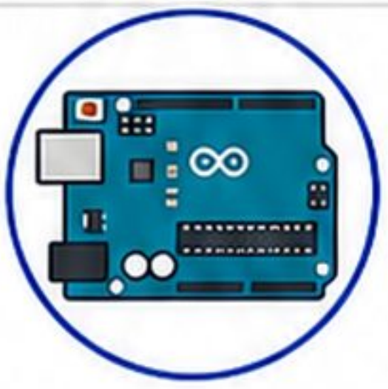
3. Operadores



4. Control de flujo



5. Funciones



6. Ejemplos prácticos

Estructura de un Programa

```
void setup() {



}

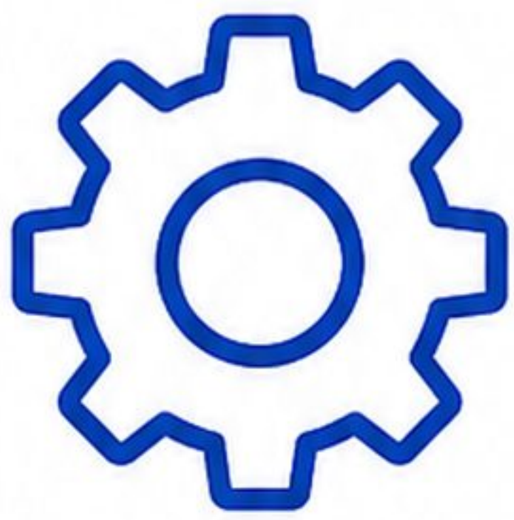
- - - - - - - - - - - - - - - -

void loop() {



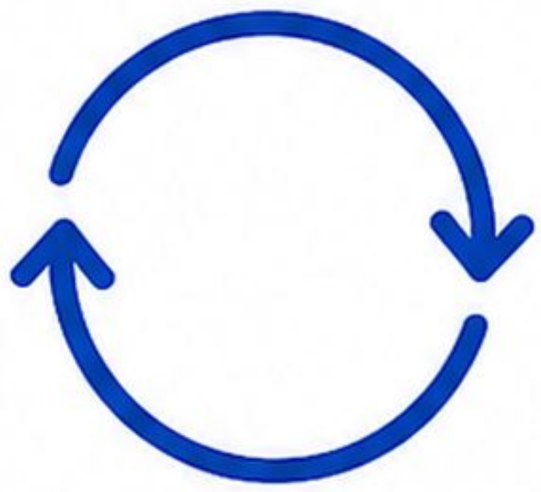
}
```

setup()



Se ejecuta **una vez**

loop()



Se repite **infinitamente**

Variables y Tipos de Datos

Los tipos de datos definen el tipo de valor que puede almacenar una variable.

123

int

Número entero.

Almacena valores enteros,
por ejemplo: -10, 0, 25, 100.

3,14

float

Número decimal (coma flotante).

Almacena números con decimales,
por ejemplo: 3,14; -0,5; 2,718.

A

char

Carácter.

Almacena un solo carácter entre
comillas simples, por ejemplo: 'A'.

boolean

Verdadero o falso.

Almacena valores lógicos:
true (verdadero) o false (falso).

Ejemplo de declaración

```
int ledPin = 13;
```

Operadores Aritméticos

Suma

7 + 3 = 10



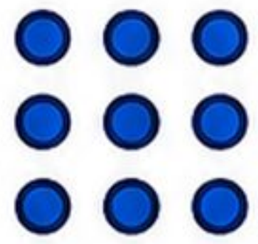
Resta

7 - 3 = 4



Multiplicación

7 * 3 = 21



División (entera)

7 / 3 = 2

(división entera)



Módulo

7 % 3 = 1



> **Cuidado con la división entera**

Operadores de Comparación

==	5 == 5 (true)	✓
!=	5 != 3 (true)	✓
<	3 < 5 (true)	✓
>	5 > 3 (true)	✓
<=	3 <= 5 (true)	✓
>=	5 >= 5 (true)	✓

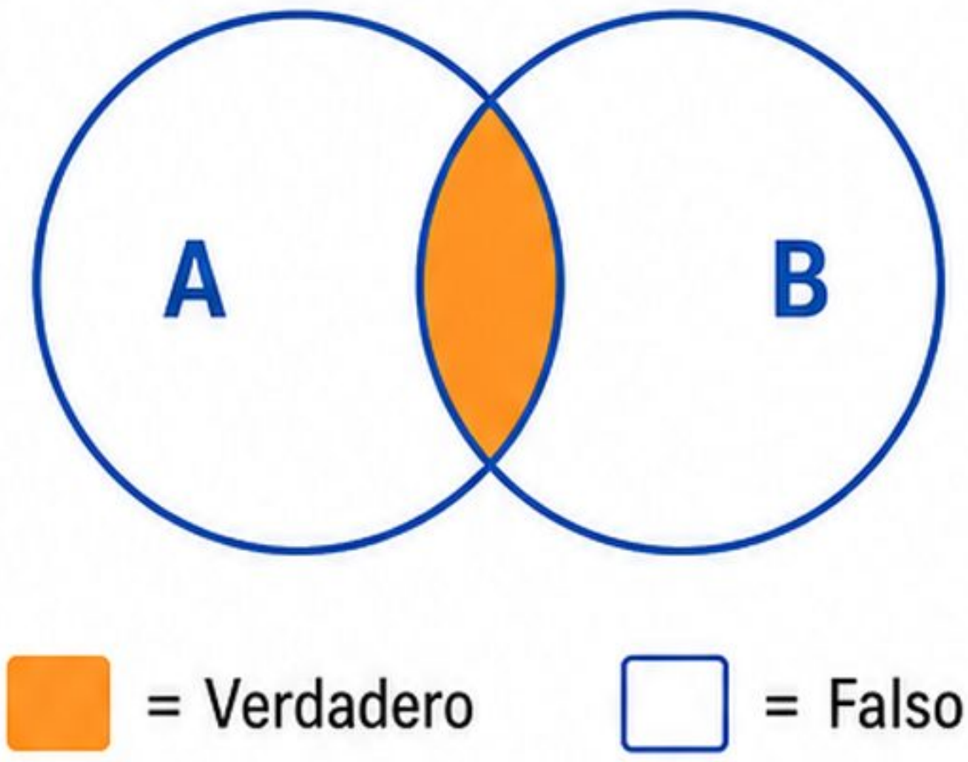
✓ (true) = Verdadero ✗ (false) = Falso

Operadores Lógicos

AND (&&)

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

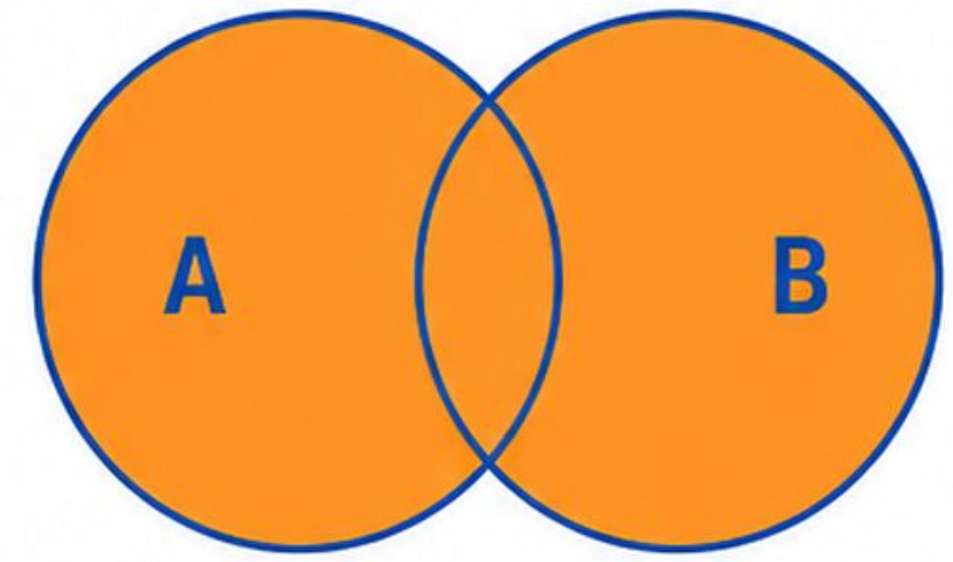
A && B es verdadero
solo cuando **A** y **B** son verdaderos



OR (||)

A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

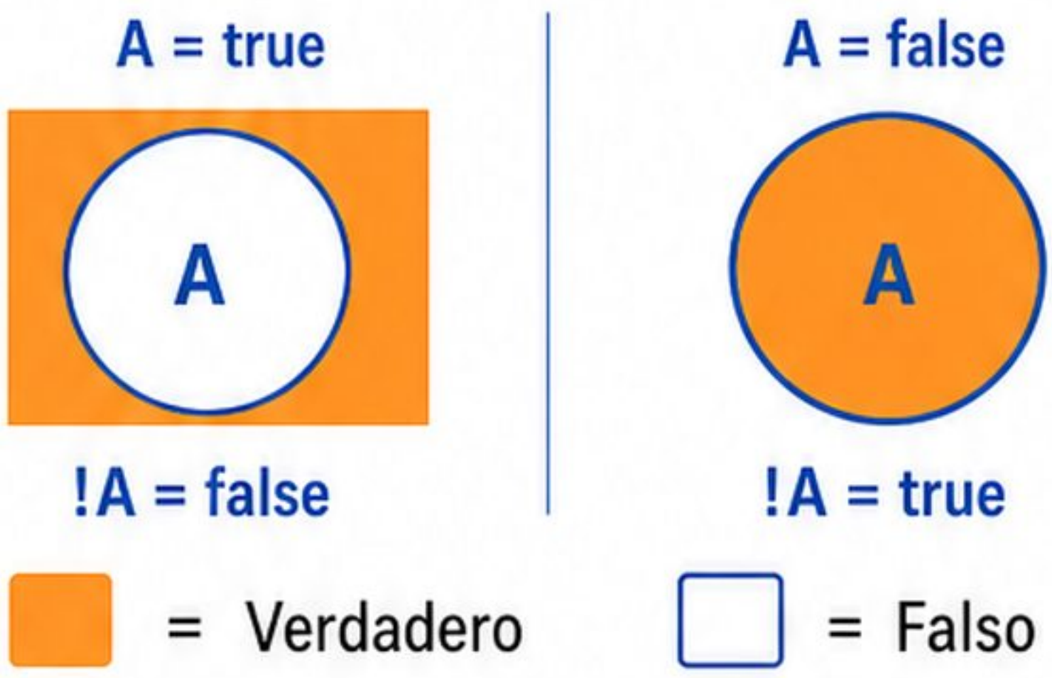
A || B es verdadero
cuando al menos una de las dos
proposiciones es verdadera



NOT (!)

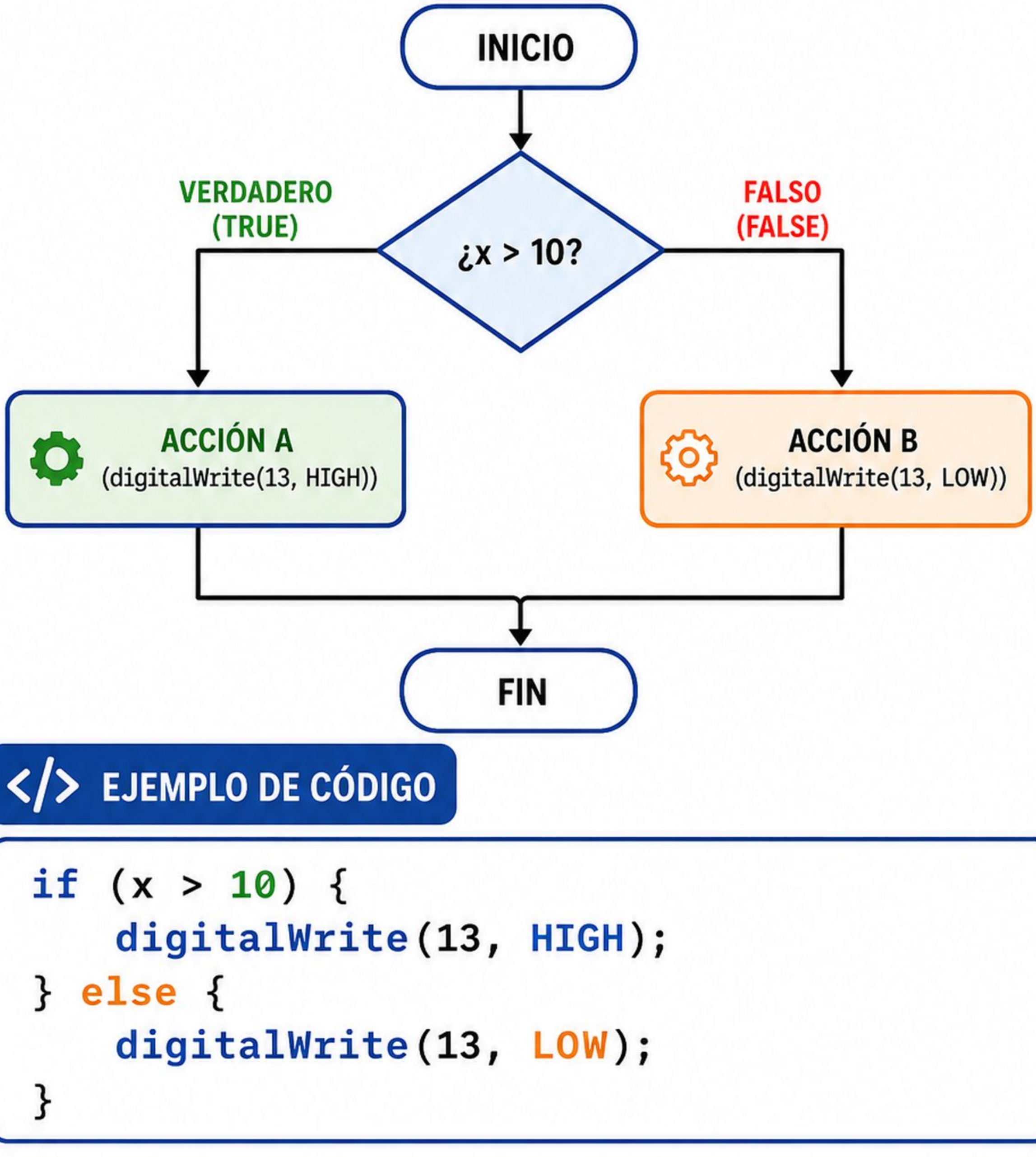
A	!A
true	false
false	true

!A invierte el valor de verdad
de la proposición **A**



Estructuras de Control: if/else

DIAGRAMA DE FLUJO



</> EJEMPLO DE CÓDIGO

```
if (x > 10) {
    digitalWrite(13, HIGH);
} else {
    digitalWrite(13, LOW);
}
```

DIAMANTE (DECISIÓN): Evalúa una condición y determina el flujo del programa.

ACCIONES: Instrucciones que se ejecutan según el resultado de la condición.

Estructuras de Control: for

1. INICIALIZACIÓN

Se ejecuta una vez al inicio del bucle.

2. CONDICIÓN

¿Se cumple la condición?

NO

El bucle termina.

SÍ

3. INCREMENTO

Se ejecuta al final de cada iteración para actualizar la variable de control.

EJEMPLO DE CÓDIGO

```
for (int i = 0; i < 10; i++) {
    Serial.println(i);
}
```

1. Inicialización: `int i = 0`
2. Condición: `i < 10`
3. Incremento: `i++`

VISUALIZACIÓN DEL BUCLE

`i` empieza en 0

0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

Mientras `i < 10`, se ejecuta el bloque de código.

El bucle se repite hasta que la condición sea falsa (`i` ya no sea menor que 10).

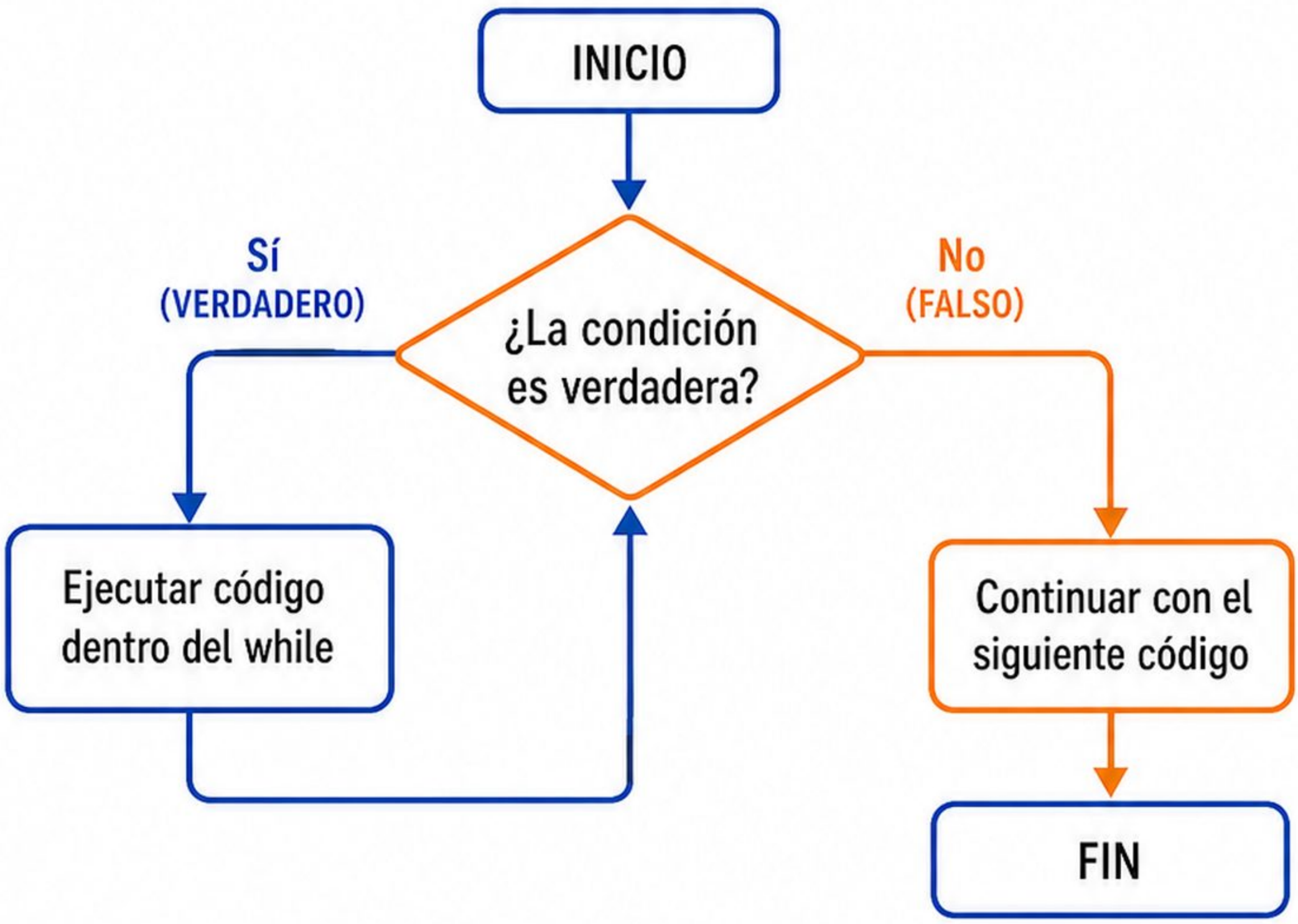
Estructuras de Control: while

SINTAXIS

```
while (condition) {
    // code
}
```

El bloque de código se ejecuta mientras la condición sea **VERDADERA**.

DIAGRAMA DE FLUJO



EJEMPLO

```
while (digitalRead(2) == LOW) {
    delay(100);
}
```

BUCLE (LOOP)

El código se repite una y otra vez mientras la condición sea verdadera.

Funciones:
Declaración y Uso

Las funciones permiten **organizar** el código, **reutilizarlo** y **simplificar** programas.

Declaración

Es **definir** la estructura de la función. Aquí se indica qué hará la función.

Ejemplo:

```
1  void setup() {
2
3  }
```

Se usa la palabra clave, el nombre de la función y llaves {}.

Llamada

Es **usar** la función en el programa. Se ejecuta el código que contiene.

Ejemplo:

```
1  setup();
2
3
```

Se escribe el nombre de la función seguido de paréntesis y punto y coma.

En resumen:
Declaras la función una vez y puedes llamarla las veces que necesites.

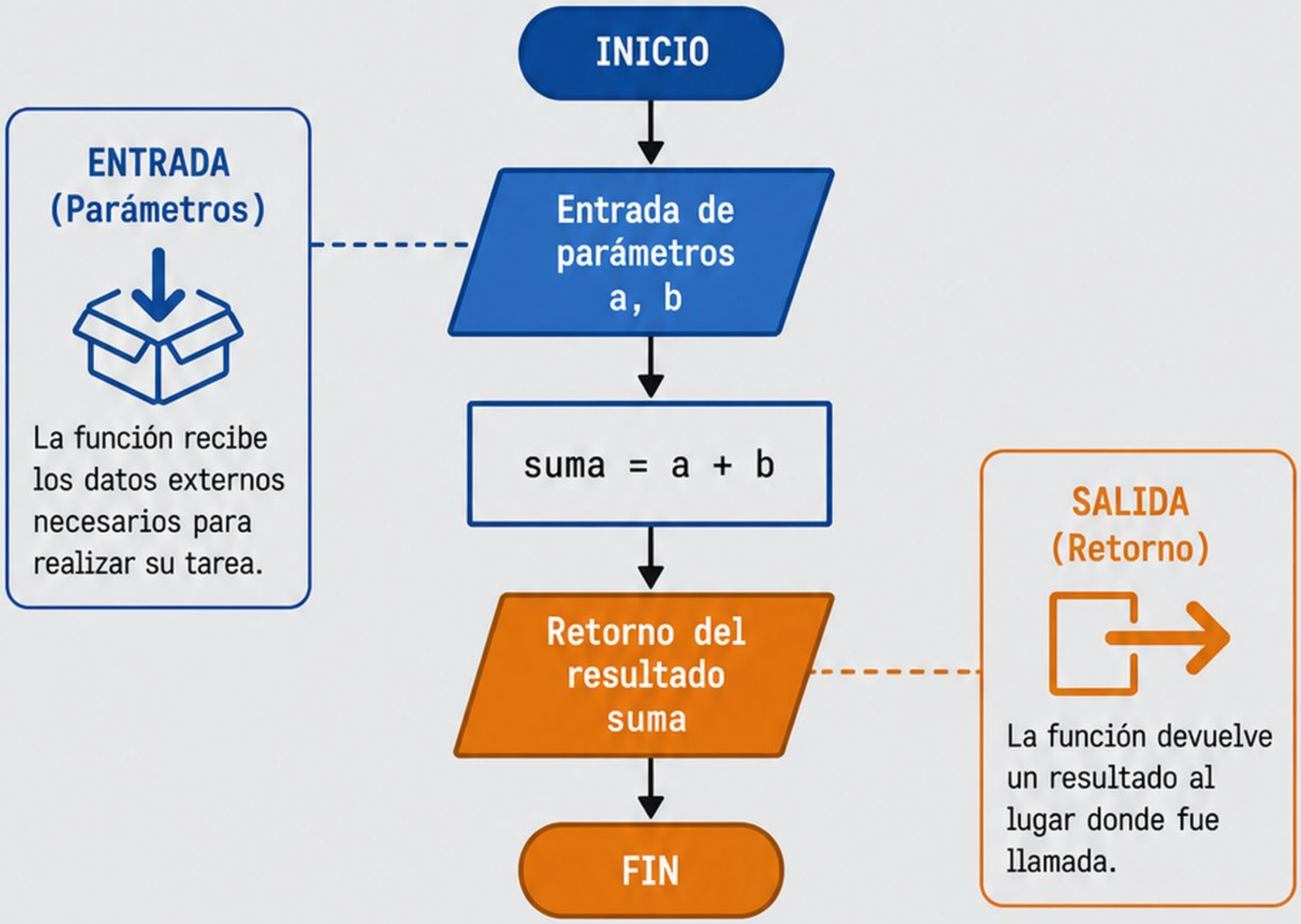
Funciones con Parámetros y Retorno

Las funciones pueden recibir datos (**parámetros**) y devolver un resultado (**retorno**).

EJEMPLO DE FUNCIÓN

```
int sumar(int a, int b) {
    return a + b;
}
```

DIAGRAMA DE FLUJO



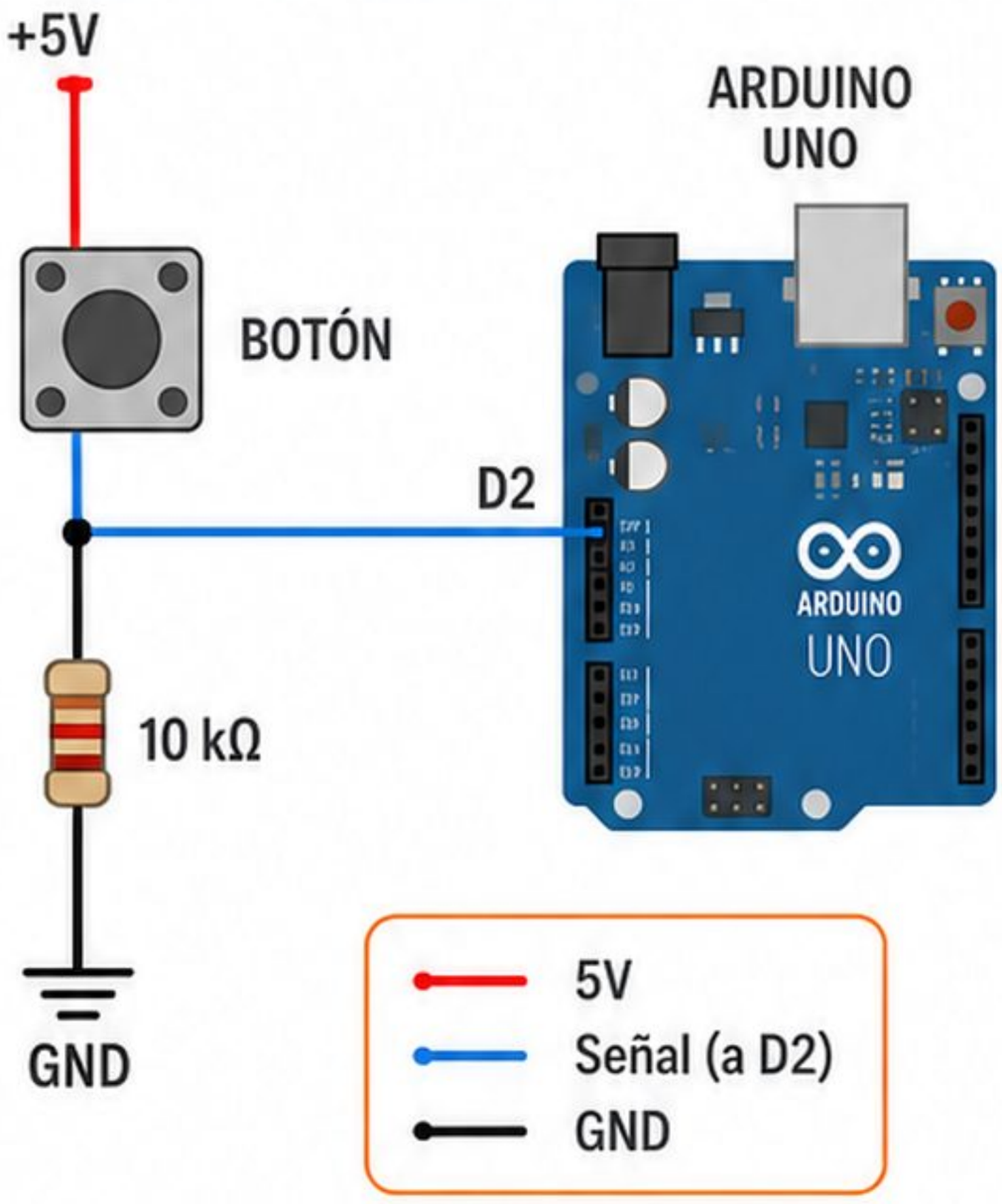
RESUMEN La función ‘sumar’ recibe dos enteros (a y b), calcula su suma y devuelve el resultado al programa que la invocó.

Entradas Digitales: Botones

Un botón es una entrada digital que permite detectar dos estados: **PRESIONADO (HIGH)** o **NO PRESIONADO (LOW)**.

Usamos una resistencia **pull-down** para asegurar un LOW estable cuando el botón no está presionado.

ESQUEMA DE CONEXIÓN



CÓMO FUNCIONA

- **Cuando el botón NO está presionado:** la resistencia pull-down mantiene la entrada en LOW (0).
- **Cuando el botón está presionado:** conecta la entrada a +5V, leyendo HIGH (1).



CÓDIGO DE EJEMPLO

```
int boton = 2;

void setup() {
  pinMode(boton, INPUT);
  Serial.begin(9600);
}

void loop() {
  if (digitalRead(boton) == HIGH) {
    Serial.println("Presionado");
  }
}
```

Lee el estado del botón con `digitalRead()`.

Si el botón está presionado (HIGH), imprime "Presionado" en el Monitor Serie.

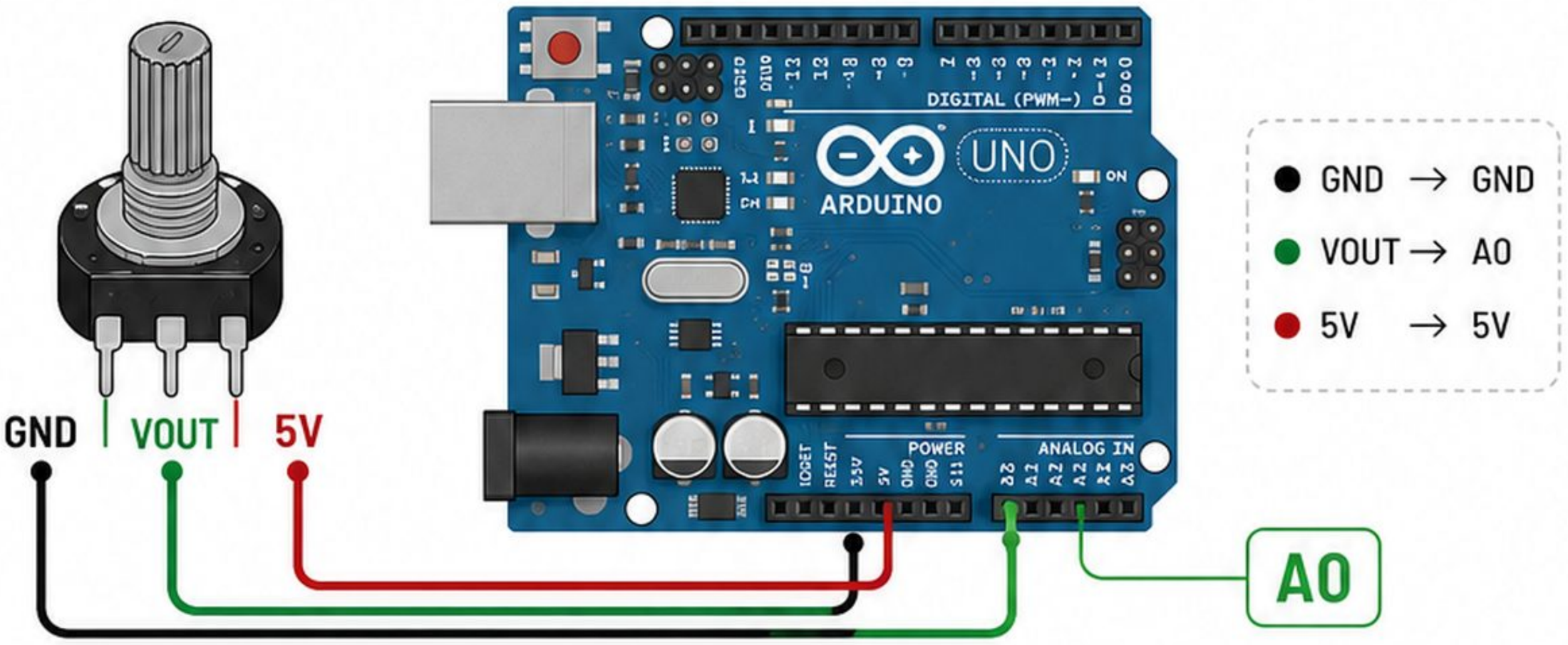
TIP: La resistencia pull-down asegura que la entrada esté en LOW cuando el botón no está presionado, evitando lecturas erráticas.



Entradas Analógicas: Potenciómetro

Un potenciómetro entrega un voltaje variable entre **0V** y **5V**.
Arduino lo lee por un pin analógico (**A0**) y lo convierte a un valor entre **0** y **1023**.

DIAGRAMA DE CONEXIÓN



CÓDIGO ARDUINO

```
int pot = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(pot);
  Serial.println(valor);
  delay(100);
}
```

¿QUÉ VALORES LEE?

Arduino convierte el voltaje leído en un número entero de 10 bits.

0 (0V) — ESCALA ANALÓGICA 10 BITS — **1023** (5V)

0 ——— 1023

Ejemplos de lectura:

- 0 ≈ 0V
- 512 ≈ 2.5V
- 1023 ≈ 5V

Valor leído (0 a 1023)

TIP:
Puedes girar el potenciómetro y observar cómo cambia el valor en el Monitor Serie. ¡Así funciona una entrada analógica!

Salidas PWM: Control de Brillos

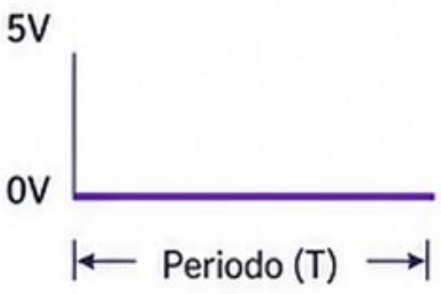
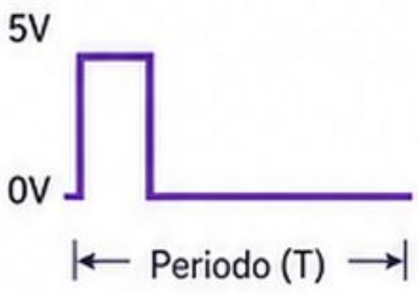
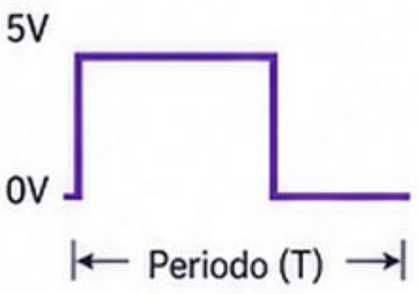
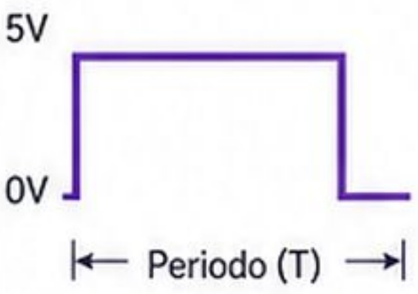
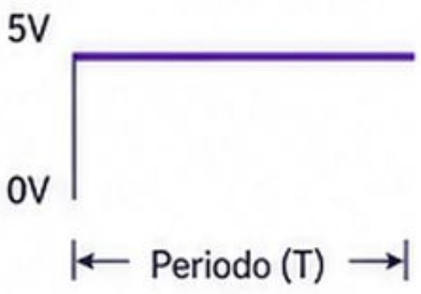
¿QUÉ ES PWM?

PWM (Modulación por Ancho de Pulso) es una técnica que consiste en encender y apagar muy rápido una señal digital. Al variar el tiempo que la señal permanece **ENCENDIDA** (en alto) en cada ciclo, podemos controlar el brillo aparente de un LED.

La frecuencia es tan alta que el LED no parpadea, y nuestro ojo percibe un brillo estable proporcional al ciclo de trabajo.

CICLO DE TRABAJO (DUTY CYCLE)

El ciclo de trabajo es el porcentaje de tiempo que la señal está en **ALTO** (encendida) durante cada ciclo.

0%	25%	50%	75%	100%
Siempre apagado	Bajo	Medio	Alto	Siempre encendido
				
Brillo: 0%	Brillo: 25%	Brillo: 50%	Brillo: 75%	Brillo: 100%

EJEMPLO DE CÓDIGO (ARDUINO)

```
1  int led = 9;
2
3  void setup() {
4    pinMode(led, OUTPUT);
5  }
6
7  void loop() {
8    for (int i = 0; i <= 255; i++) {
9      analogWrite(led, i);
10     delay(10);
11   }
}
```

analogWrite(pin, valor) genera una señal PWM en el pin especificado.

0-255

El valor va de 0 a 255:
0 → 0% de ciclo de trabajo (apagado)
255 → 100% de ciclo de trabajo (encendido)

En este ejemplo, el LED en el pin 9 aumenta su brillo suavemente desde apagado (0) hasta máximo (255) y se repite continuamente.

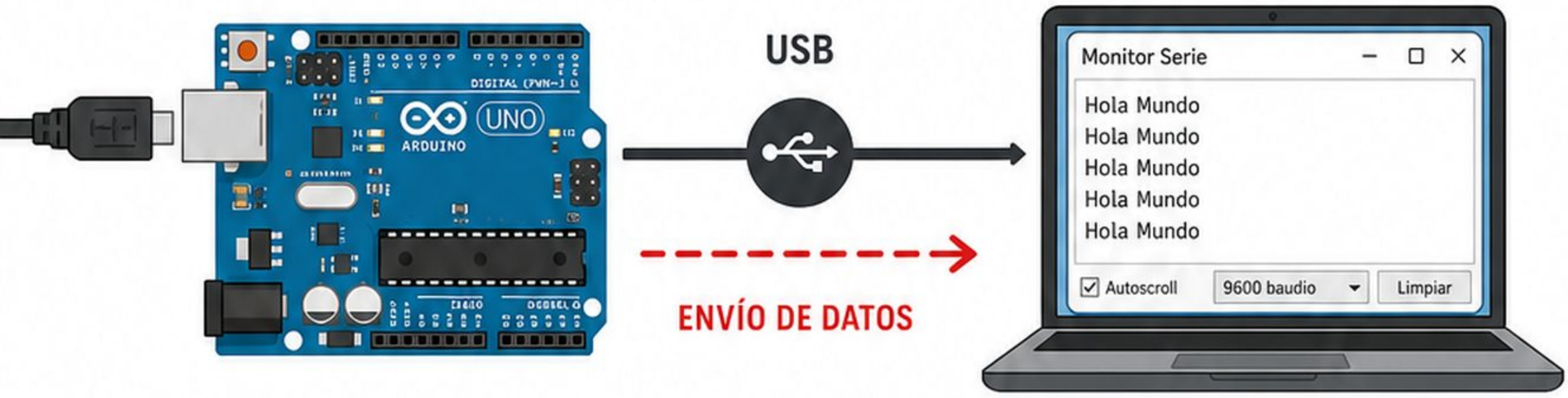
IDEAL PARA:

- Control de brillo de LEDs
- Control de velocidad de motores
- Generación de tonos y audio
- Control de potencia en dispositivos

TIP: PWM no cambia el voltaje, cambia el tiempo que la señal está encendida. Esto lo hace muy eficiente para controlar dispositivos.

COMUNICACIÓN SERIAL:
ENVIAR DATOS

La comunicación serial permite enviar datos desde el Arduino a una computadora a través de **USB**.



CÓDIGO

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("Hola Mundo");
  delay(1000);
}
```

`Serial.begin(9600);`
Inicia la comunicación serial a 9600 baudios.

`Serial.println("Hola Mundo");`
Envía el texto “Hola Mundo” y agrega un salto de línea.

`delay(1000);`
Espera 1000 milisegundos (1 segundo) antes de enviar el siguiente mensaje.

¿CÓMO FUNCIONA?

Genera los datos → Los datos viajan como señales eléctricas → La computadora recibe y muestra los datos

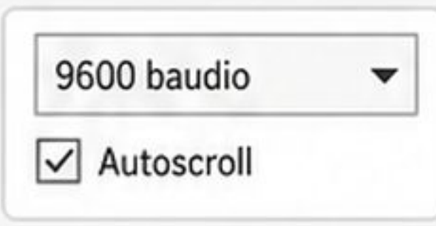
ONDA DE DATOS



Los datos se transmiten como niveles de voltaje: **ALTO (5V)** y **BAJO (0V)**.

RECUERDA

Para ver los datos, abre el Monitor Serie en el IDE de Arduino, selecciona 9600 baudios y asegúrate de que esté conectada la placa.



Comunicación Serial:
Recibir Datos

El programa escucha datos que llegan por el puerto serie, los lee y los muestra en el monitor serie.

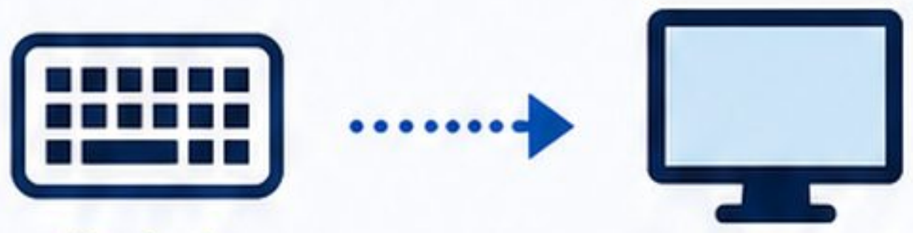
Código Arduino

```
1   void setup() {
2     Serial.begin(9600);
3   }
4
5   void loop() {
6     if (Serial.available() > 0) {
7       char dato = Serial.read();
8       Serial.print("Recibido: ");
9       Serial.println(dato);
10    }
11  }
12
13
14
```

¿Qué hace este programa?

- Inicializa la comunicación serial a 9600 baudios.
- En cada iteración, verifica si hay datos disponibles.
- Si hay datos, lee un carácter.
- Muestra el dato recibido en el monitor serie, precedido por “Recibido: ”.

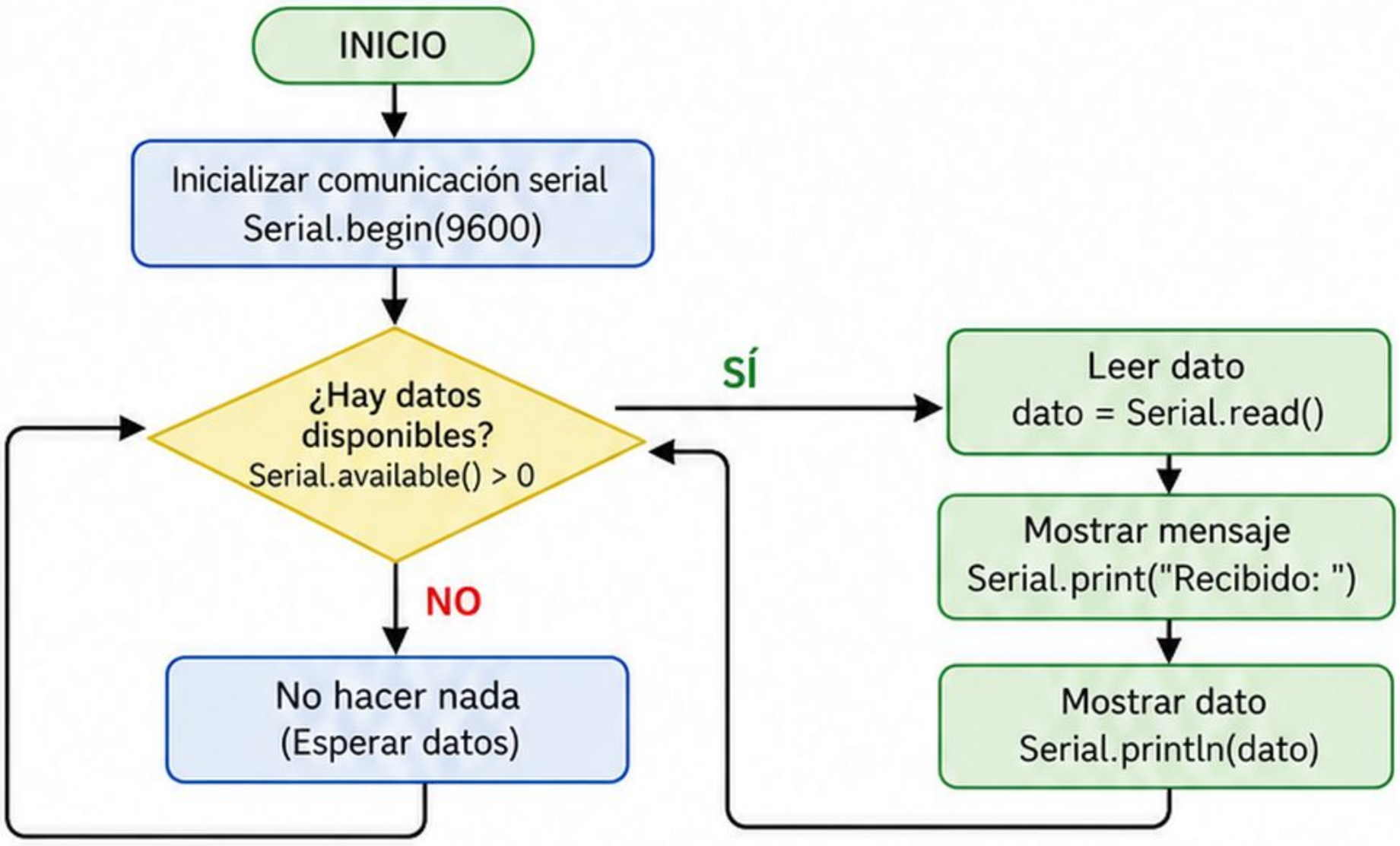
Herramientas



Teclado
(Envía datos)

Monitor Serie
(Muestra datos)

Diagrama de flujo de recepción



TIP:
Abre el Monitor Serie, selecciona 9600 baudios y envía caracteres desde el teclado.

Ejemplo de salida en el Monitor Serie:

```
Recibido: A
Recibido: 1
Recibido: ?
```

Variables Globales y Locales

Las variables pueden tener diferentes alcances según dónde se declaren.



Globales

Declaradas fuera de las funciones, accesibles desde cualquier parte del programa.



Locales

Declaradas dentro de una función, solo son visibles y utilizables dentro de esa función.

Ejemplo de código

```
1   // Variable global
2   int contador = 0;
3
4   void funcion() {
5       // Variable local
6       int temp = 5;
7       temp = temp + 1;
8       // contador también es accesible aquí
9       contador = contador + 1;
10  }
```

Global: accesible en todo el programa

Local: solo visible dentro de `funcion()`

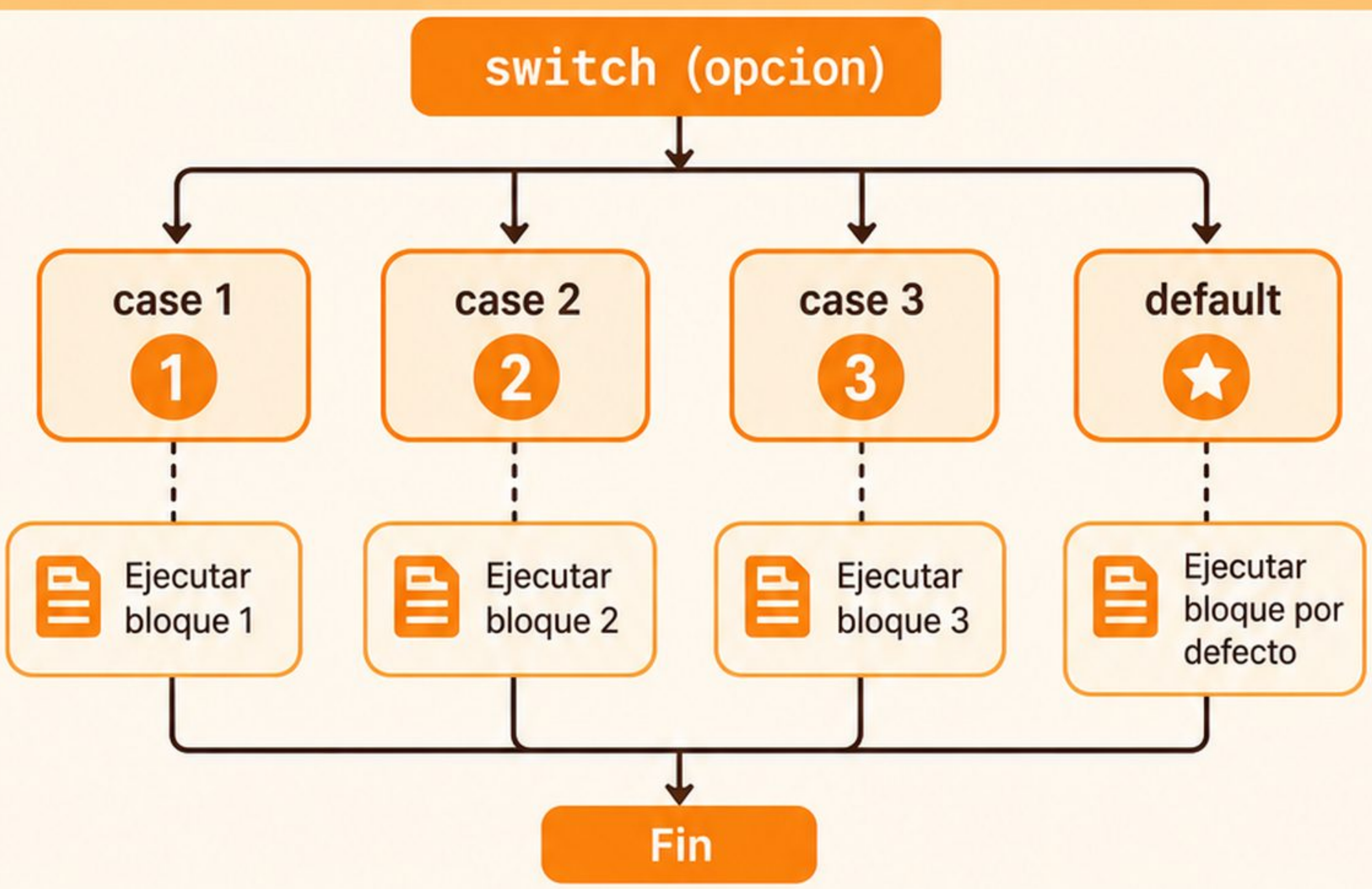
En resumen:

Usa variables globales para datos compartidos en todo el programa y variables locales para datos temporales dentro de funciones específicas.

Switch Case



Switch case permite elegir entre varias opciones según el valor de una variable.



Ejemplo de código

```
switch (opcion) {
  case 1:
    // instrucciones para la opción 1
    break;
  case 2:
    // instrucciones para la opción 2
    break;
  case 3:
    // instrucciones para la opción 3
    break;
  default:
    // instrucciones por defecto
}
```

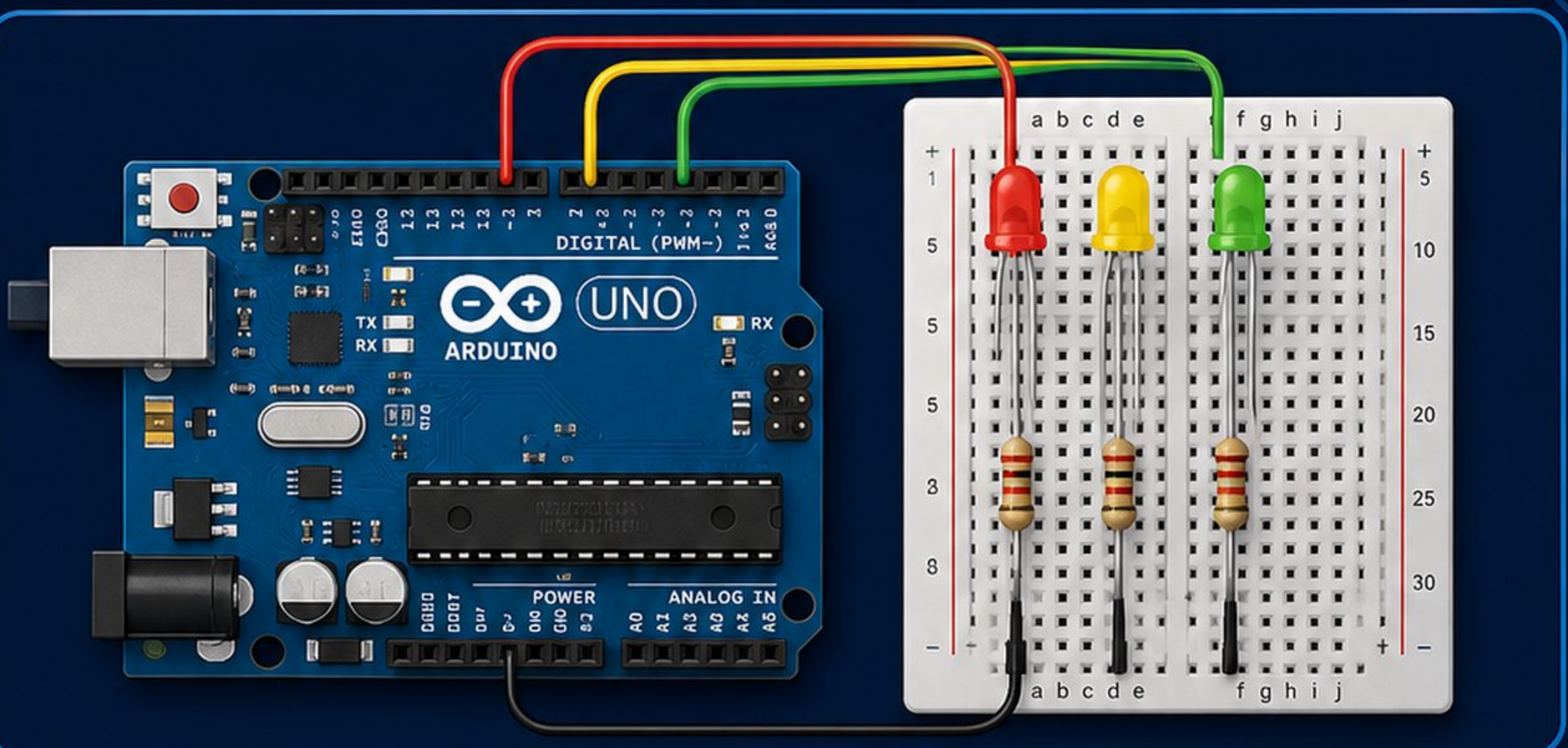
Recuerda

- Cada caso termina con **break** para evitar que se ejecuten otros casos.
- El **default** es opcional y se ejecuta si no hay coincidencias.
- Usa **switch case** cuando tengas varias alternativas discretas.

PROYECTO

Ejercicio Práctico: Semáforo

Pon en práctica lo aprendido creando un semáforo.



CÓDIGO DE EJEMPLO

```
void loop() {
  digitalWrite(rojo, HIGH);
  delay(5000);
  digitalWrite(rojo, LOW);

  digitalWrite(verde, HIGH);
  delay(5000);
  digitalWrite(verde, LOW);
  ...
}
```

MATERIALES

- 3 LEDs
- 3 resistencias
- cables
- protoboard

OBJETIVO: Controlar el encendido de los LEDs en secuencia, simulando el funcionamiento de un semáforo real.

Conclusión y Próximos Pasos

RESUMEN DE CONCEPTOS CLAVE

Funciones
Bloques de código reutilizables que realizan tareas específicas.

Parámetros
Valores que se pasan a las funciones para personalizar su comportamiento.

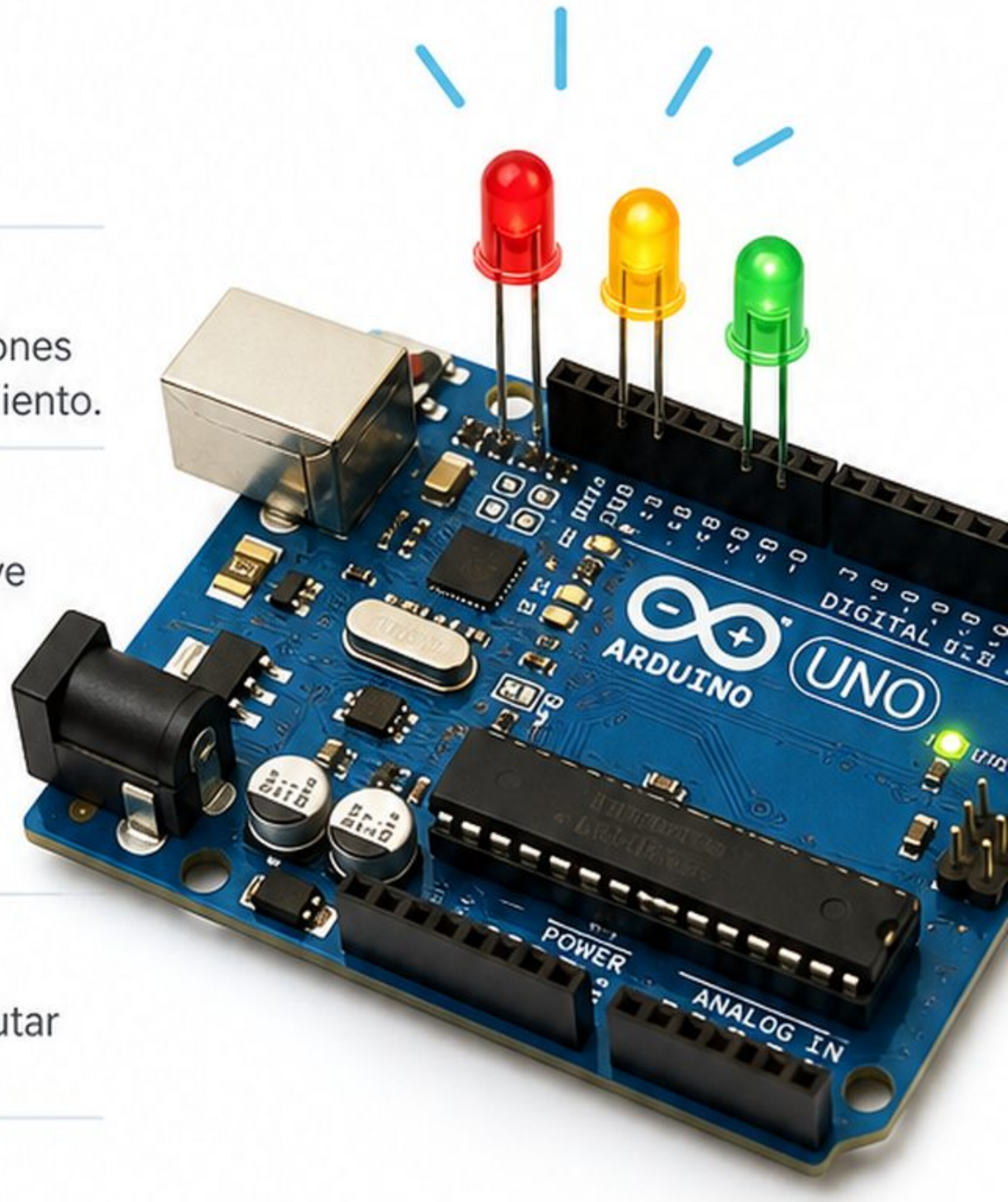
Retorno
Valores que una función devuelve después de ejecutarse.

Variables
Almacenan datos que pueden cambiar durante el programa.

if/else
Permite tomar decisiones y ejecutar código según condiciones.

Bucles
Repiten bloques de código mientras se cumpla una condición.

Switch
Selecciona una opción entre varias posibles de manera eficiente.



¡Felicidades!

Ya tienes las bases para programar Arduino.

Continúa con proyectos más complejos.

¡Sigue aprendiendo, creando y construyendo el futuro!